

**LAPORAN PRAKTIKUM PENGEMBANGAN APLIKASI BERGERAK
MANAGEMEN MAHASISWA**



Disusun oleh:

Nama : Mufti Faris Murtadho

Nim : L0124133

Kelas : B

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI DAN SAINS DATA
UNIVERSITAS SEBELAS MARET**

2026

1. Screenshot Source Code

A. Struktur Data dan Inisialisasi (Data Model)

```
4  data class Mahasiswa(  
5      val nim: String,      // String (data type), val (immutable variable)  
6      var nama: String,     // String (data type), var (mutable variable)  
7      var nilai: Int        // Int (data type)  
8  )
```

```
13 class SistemMahasiswa {  
14     private val data: MutableList<Mahasiswa> = mutableListOf()  
16 }
```

Penjelasan:

Pada bagian ini program mendefinisikan entitas utama yang akan dikelola.

a) Data Class Mahasiswa

Digunakan untuk menyimpan data NIM, Nama, dan Nilai. Di sini diimplementasikan Data Types (String, Int) serta penggunaan Variables val untuk data yang tetap (NIM) dan var untuk data yang dapat berubah (Nama, Nilai).

b) Encapsulation

Daftar mahasiswa disimpan di dalam private val data berupa MutableList, sehingga data hanya bisa dimanipulasi melalui fungsi-fungsi yang tersedia di kelas SistemMahasiswa.

B. Logika Manajemen Data (Fungsi & Nilai Tambahan)

```
21 fun tambah(vararg mahasiswas: Mahasiswa) {  
22     for (m in mahasiswas) {  
23         data.add(m)  
24     }  
25 }  
26  
27 // -----  
28 // [TAMBAHAN 2] DEFAULT ARGUMENT  
29 // Parameter pesanKosong memiliki nilai default jika tidak diisi  
30 // -----  
31 fun tampil(pesanKosong: String = "Data kosong") {  
32     if (data.isEmpty()) {  
33         println(pesanKosong)  
34     } else {  
35         data.forEachIndexed { i, m ->  
36             val nomor = i + 1  
37             println(nomor.toString() + ". " + m.nim + " - " + m.nama + " - " + m.nilai.toString())  
38         }  
39     }  
40 }  
41  
242 sistem.update(keyword = nim, namaBaru = nama, nilaiBaru = nilai)
```

Pejelasan:

Bagian ini menjelaskan bagaimana program memproses data menggunakan berbagai fitur lanjutan Kotlin:

a) Fungsi Tambah (Vararg)

Menggunakan poin tambahan Variable Argument (Vararg) agar fungsi dapat menerima banyak objek mahasiswa sekaligus dalam satu baris perintah.

b) Fungsi Tampil (Default Argument)

Menggunakan Default Argument pada parameter pesan. Jika daftar kosong, program akan menampilkan pesan bawaan tanpa perlu input manual dari pemanggil fungsi.

c) Fungsi Update (Named Argument)

Saat memanggil fungsi ini di main, digunakan Named Argument untuk memastikan argumen masuk ke parameter yang tepat (misal: `namaBaru = ...`), yang meningkatkan keterbacaan kode.

C. Penanganan Null dan Alur Kondisional

```

43 // =====
44 // [WAJIB 3] NULLABLE TYPES & [WAJIB 4] SAFE CALLS
45 // Parameter keyword adalah Nullable (String?)
46 // =====
47 fun cari(keyword: String?) {
48     // [WAJIB 4] Safe call (?)
49     val trimmed = keyword?.trim()
50
51     val key = if (trimmed != null) trimmed else ""
52
53     val hasil = data.filter {
54         it.nim.contains(key, ignoreCase = true)
55     }
56
57     if (hasil.isEmpty()) {
58         println("Tidak ditemukan")
59         return
60     }
61
62     if (hasil.size == 1) {
63         val m = hasil[0]
64         println("Ditemukan: " + m.nim + " - " + m.nama + " - " + m.nilai.toString())
65         return
66     }
67
68     println("Ditemukan lebih dari satu:")
69     hasil.forEachIndexed { i, m ->
70         val nomor = i + 1
71         println(nomor.toString() + ". " + m.nim + " - " + m.nama)
72     }
73
74     print("Pilih nomor: ")
75     val pilih = readLine()?.toIntOrNull() // Safe call
76
77     if (pilih == null || pilih !in 1..hasil.size) {
78         println("Pilihan tidak valid")
79         return
80     }
81
82     val m = hasil[pilih - 1]
83     println("Hasil final: " + m.nim + " - " + m.nama + " - " + m.nilai.toString())
84 }

```

Penjelasan:

Bagian ini sangat krusial untuk mencegah aplikasi *crash* dan menunjukkan kompleksitas program:

a) Nullable Types & Safes Calls

Semua input dari `readLine()` atau parameter pencarian didefinisikan sebagai Nullable (`String?`). Program menggunakan Safe Call (`?.`) untuk memproses data tersebut hanya jika tidak bernilai null, sehingga aplikasi bebas dari *NullPointerException*.

b) If Expression

Digunakan bukan hanya sebagai alur kontrol, tapi sebagai ekspresi yang menghasilkan nilai (misalnya untuk menentukan objek target yang akan dihapus atau diupdate), yang merupakan ciri khas pemrograman fungsional di Kotlin.

c) Logika Filter & Pilihan

Program tidak langsung menghapus data jika ditemukan banyak kecocokan NIM, melainkan memfilter hasil dan meminta pengguna memilih nomor urut tertentu. Ini menunjukkan logika yang lebih kompleks dibanding CRUD biasa.

D. Antarmuka Pengguna (Main Function)

```
181 fun main() {
182     val sistem = SistemMahasiswa()
183
184     while (true) {
185         println("===== MENU =====")
186         println("1. Tambah")
187         println("2. Tampil")
188         println("3. Cari")
189         println("4. Update")
190         println("5. Hapus")
191         println("6. Keluar")
192         print("Pilih: ")
193
194         val input = readLine()
195         val pilih = if (input != null) input.toIntOrNull() else null
196
197         when (pilih) {
198 >             1 -> { ...
212             }
213
214 >             2 -> { ...
217             }
218
219 >             3 -> { ...
223             }
224
225 >             4 -> { ...
243             }
244
245 >             5 -> { ...
250             }
251
252             6 -> return
253
254             else -> println("Input tidak valid")
255         }
256     }
257 }
```

Penjelasan:

a) Looping Menu

Fungsi main menjalankan perulangan while(true) agar pengguna dapat mengelola data secara terus-menerus hingga memilih opsi keluar.

b) Validasi Input

Setiap input angka diverifikasi menggunakan `toIntOrNull()` dan diproses dengan `If Expression` untuk memastikan bahwa input yang salah tidak merusak jalannya program.

E. Poin Implementasi Wajib (Mandatory)

a) Data Types & Variables

```
4 data class Mahasiswa(  
5     val nim: String,      // String (data type), val (immutable variable)  
6     var nama: String,     // String (data type), var (mutable variable)  
7     var nilai: Int        // Int (data type)  
8 )
```

Baris ini menunjukkan penggunaan tipe data standar (`String`, `Int`) serta perbedaan antara variabel yang tidak bisa diubah (`val`) dan yang bisa diubah (`var`).

b) Functions (Selain Main)

```
21 > fun tambah(vararg mahasiswa: Mahasiswa) { ...  
25     }  
26  
27     // -----  
28     // [TAMBAHAN 2] DEFAULT ARGUMENT  
29     // Parameter pesanKosong memiliki nilai default jika tidak diisi  
30     // -----  
31 > fun tampil(pesanKosong: String = "Data kosong") { ...  
41     }  
42  
43     // =====  
44     // [WAJIB 3] NULLABLE TYPES & [WAJIB 4] SAFE CALLS  
45     // Parameter keyword adalah Nullable (String?)  
46     // =====  
47 > fun cari(keyword: String?) { ...  
84     }
```

Program mendefinisikan beberapa fungsi di dalam kelas `SistemMahasiswa` untuk mengolah logika data secara terpisah dari fungsi utama.

c) Nullable Types

```
47 > fun cari(keyword: String?) { ...  
84     }
```

Penggunaan tanda tanya (?) pada tipe data `String?` menandakan bahwa variabel tersebut diperbolehkan bernilai `null`.

d) Safe Calls

```
val trimmed = keyword?.trim()
```

```
val pilih = readLine()?.toIntOrNull() // Safe call
```

Operator `?.` memastikan bahwa fungsi `trim()` atau `toIntOrNull()` hanya akan dijalankan jika objeknya tidak `null`, mencegah terjadinya *crash*.

e) If Expression

```
99 // =====
100 // [WAJIB 5] IF EXPRESSION (Mengembalikan nilai ke variabel)
101 // =====
102 val target = if (hasil.size == 1) {
103     |   hasil[0]
104 } else {
105     |   println("Ditemukan lebih dari satu:")
106     |   hasil.forEachIndexed { i, m ->
107     |       |   val nomor = i + 1
108     |       |   println(nomor.toString() + ". " + m.nim + " - " + m.nama)
109     |       }
110
111     |   print("Pilih nomor yang akan dihapus: ")
112     |   val pilih = readLine()?.toIntOrNull()
113
114     |   if (pilih == null || pilih !in 1..hasil.size) {
115     |       |   println("Pilihan tidak valid")
116     |       |   return
117     |       }
118     |   hasil[pilih - 1]
119 }
120
```

Di sini if tidak hanya digunakan sebagai pengontrol alur, tetapi langsung menghasilkan nilai yang dimasukkan ke dalam variabel target.

F. Poin Implementasi Tambahan

a) Named Argument

```
238 // -----
239 // [TAMBAHAN 1] NAMED ARGUMENT
240 // Menyebutkan nama parameter secara eksplisit saat pemanggilan
241 // -----
242 sistem.update(keyword = nim, namaBaru = nama, nilaiBaru = nilai)
```

Saat memanggil fungsi update, nama parameter dituliskan secara eksplisit (keyword =, namaBaru =) agar urutan argumen lebih jelas dan mudah dibaca.

b) Default Argument

```
27 // -----
28 // [TAMBAHAN 2] DEFAULT ARGUMENT
29 // Parameter pesanKosong memiliki nilai default jika tidak diisi
30 // -----
31 fun tampil(pesanKosong: String = "Data kosong") {
32     |   if (data.isEmpty()) {
33     |       |   println(pesanKosong)
34     |       } else {
35     |       |   data.forEachIndexed { i, m ->
36     |       |       |   val nomor = i + 1
37     |       |       |   println(nomor.toString() + ". " + m.nim + " - " + m.nama + " - " + m.nilai.toString())
38     |       |       }
39     |       }
40     }
41 }
42
```

Parameter `pesanKosong` memiliki nilai bawaan. Jika saat memanggil `sistem.tampil()` kita tidak mengisi apa pun, maka secara otomatis akan menggunakan teks "Data kosong".

c) Variable Argument (Vararg)

```
17 // -----  
18 // [TAMBAHAN 3] VARIABLE ARGUMENT (VARARG)  
19 // Memungkinkan memasukkan lebih dari 1 mahasiswa sekaligus  
20 // -----  
21 fun tambah(vararg mahasiswa: Mahasiswa) {  
22     for (m in mahasiswa) {  
23         data.add(m)  
24     }  
25 }
```

Kata kunci `vararg` memungkinkan fungsi `tambah` untuk menerima jumlah objek `Mahasiswa` yang dinamis (bisa satu, dua, atau banyak sekaligus).

2. Screenshot Aplikasi

A. Tampilan Awal Program

```
PS D:\KULIAH\SEMESTER 4\Prak PAB\PPAB-01_L0124133_MuftiFarisMurtadho> cd "d:\KULIAH\SEMESTER 4\Prak PAB\PPAB-01_L0124133_MuftiFarisMurtadho\" ; if ($?) { kotlinc main.kt -include-r
untime -d main.jar } ; if ($?) { java -jar main.jar }
===== MENU =====
1. Tambah
2. Tampil
3. Cari
4. Update
5. Hapus
6. Keluar
Pilih: |
```

Program berjalan dalam loop sehingga menu terus muncul sampai user memilih keluar.

B. Proses Tambah Data

```
Pilih: 1
NIM: L0124133
Nama: Mufti Faris
Nilai: 90
===== MENU =====
1. Tambah
2. Tampil
3. Cari
4. Update
5. Hapus
6. Keluar
Pilih: |
```

User memasukkan NIM, nama dan nilai. Data kemudian disimpan ke dalam list.

C. Proses Tampil Data

```
Pilih: 2
1. L0124133 - Mufti Faris - 90
2. L0124999 - Moros - 99
===== MENU =====
1. Tambah
2. Tampil
3. Cari
4. Update
5. Hapus
6. Keluar
Pilih: |
```

Menampilkan seluruh data mahasiswa yang sudah dimasukkan.

```
Pilih: 2
Data kosong
===== MENU =====
1. Tambah
2. Tampil
3. Cari
4. Update
5. Hapus
6. Keluar
Pilih: |
```

Jika belum ada data, akan muncul pesan “Data kosong”.

D. Proses Pencarian Data

```
Pilih: 3
Keyword NIM: L0124133
Ditemukan: L0124133 - Mufti Faris - 90
===== MENU =====
1. Tambah
2. Tampil
3. Cari
4. Update
5. Hapus
6. Keluar
Pilih: |
```

```
Pilih: 3
Keyword NIM: L0124
Ditemukan lebih dari satu:
1. L0124133 - Mufti Faris
2. L0124999 - Moros
Pilih nomor: 1
Hasil final: L0124133 - Mufti Faris - 90
===== MENU =====
1. Tambah
2. Tampil
3. Cari
4. Update
5. Hapus
6. Keluar
Pilih: |
```

User memasukkan sebagian atau seluruh NIM lalu sistem akan menampilkan hasilnya jika cocok dan juga jika memasukkan NIM hanya sebagian dan muncul lebih dari satu user dapat memilih datanya (double check).

E. Proses Update Data

```
Pilih: 4
NIM: L0124
Nama baru: Mufti Faris Murtadho
Nilai baru: 91
Ditemukan lebih dari satu:
1. L0124133 - Mufti Faris
2. L0124999 - Moros
Pilih nomor yang akan diupdate: 1
Yakin update L0124133 menjadi [Mufti Faris Murtadho, 91]? (y/n): y
Data diupdate
===== MENU =====
1. Tambah
2. Tampil
3. Cari
4. Update
5. Hapus
6. Keluar
Pilih: 
```

User memilih data berdasarkan NIM bisa sebagian juga lalu memasukkan data baru nama dan nilai setelah itu user akan di minta konfirmasi ulang sebelum update dengan y/n.

F. Proses Hapus Data

```
Pilih: 5
NIM dihapus: L0124
Ditemukan lebih dari satu:
1. L0124133 - Mufti Faris Murtadho
2. L0124999 - Moros
Pilih nomor yang akan dihapus: 2
Apakah anda yakin menghapus L0124999? (y/n): y
Data dihapus
===== MENU =====
1. Tambah
2. Tampil
3. Cari
4. Update
5. Hapus
6. Keluar
Pilih: 
```

```
Pilih: 5
NIM dihapus: L0124
Ditemukan lebih dari satu:
1. L0124133 - Mufti Faris Murtadho
2. L0124999 - Moros
Pilih nomor yang akan dihapus: 2
Apakah anda yakin menghapus L0124999? (y/n): n
Dibatalakan
===== MENU =====
1. Tambah
2. Tampil
3. Cari
4. Update
5. Hapus
6. Keluar
Pilih: 
```

User mencari data menggunakan NIM, memilih jika memang memasukkan NIM hanya sebagian lalu sistem akan meminta konfirmasi ulang ketika ingin menghapus datanya dengan y/n.

G. Keluar Program

```
===== MENU =====
1. Tambah
2. Tampil
3. Cari
4. Update
5. Hapus
6. Keluar
Pilih: 6
PS D:\KULIAH\SEMESTER 4\Prak PAB\PPAB-01 L0124133 MuftiFarisMurtadho> 
```

Jika user memilih opsi 6 maka program akan berhenti sepenuhnya.